

Máquina de pelotas

Madina ha inventado una máquina de pelotas para entretener a los participantes de la IOI. La estructura interna de la máquina es la siguiente:

- La máquina consta de **nodos** N , indexados desde 0 hasta $N - 1$. El nodo $N - 1$ se denomina **raíz**.
- Para cada nodo u con $0 \leq u < N - 1$, el **padre** del nodo u es un nodo $P[u]$ con $P[u] > u$, y el nodo u es un **hijo** de $P[u]$. La raíz no tiene padre. Un nodo sin hijos se denomina **hoja**.

No conoces N ni el padre $P[u]$ de ningún nodo u . En cambio, Madina te indica M , el número de hojas, y que los índices de las hojas son $0, 1, \dots, M - 1$. Tu tarea consiste en determinar la estructura interna de la máquina mediante su funcionamiento.

Cada nodo de la máquina puede contener como máximo una **bola**, y cada bola tiene un **valor** que es un entero no negativo. Decimos que un nodo tiene valor x si contiene una bola con valor x . Un nodo está **ocupado** si contiene una bola; en caso contrario, está **libre**. Inicialmente, todos los nodos están libres.

Se pueden realizar dos tipos de operaciones:

- **insert(U, X)**: escribir el valor X en una nueva bola e intentar insertarla en la hoja U .
 - Si la hoja U está ocupada, la operación devuelve `false` sin insertar la bola.
 - Si la hoja U está libre, la bola se coloca en el nodo U . A continuación, la bola se desplaza repetidamente hacia el nodo padre de su nodo actual, siempre que dicho nodo padre esté libre. El movimiento se detiene cuando la bola llega a la raíz o a un nodo cuyo padre esté ocupado. La operación devuelve `true`.
- **collect()**: si la raíz está libre (lo que significa que la máquina está vacía), `collect` devuelve un arreglo vacío. De lo contrario, el procedimiento recursivo `traverse` descrito a continuación recopila los valores de todas las bolas que se encuentran actualmente en la máquina en un arreglo S . Inicialmente, el arreglo S está vacío y se llama a `traverse( $N - 1$ )`.

```
traverse(u):

    añade el valor de la bola del nodo u al final de S

    sea c[u] la lista de los hijos ocupados de u

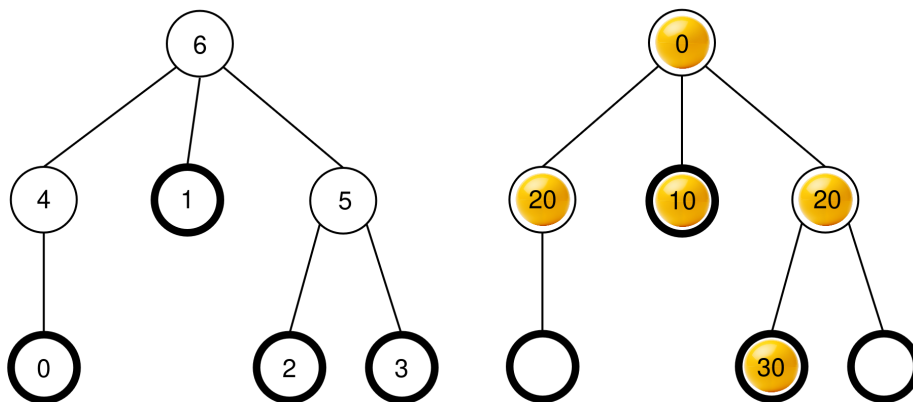
    ordena c[u] en orden no decreciente según los valores de las bolas en esos nodos (si varios nodos tienen el
    mismo valor, se ordenan por su índice)

    para cada v en c[u]:

        traverse(v)
```

Una vez finalizado este procedimiento, **todas las bolas se retiran** de la máquina, y `collect` devuelve S .

Por ejemplo, consideremos una máquina con $N = 7$ nodos y un arreglo de padres $P = [4, 6, 5, 5, 6, 6]$. La imagen de la izquierda muestra la máquina vacía con los índices de los nodos, mientras que la de la derecha muestra una posible configuración de las bolas tras una secuencia de operaciones `insert` (esta secuencia se detalla en la sección «Ejemplo»).



Cuando se llama a `collect()`, la raíz (nodo 6) está ocupada, por lo que S se inicializa como $S = []$ y se llama a `traverse(6)`.

traverse(6) : el nodo 6 tiene el valor 0 ; al añadirlo a S se obtiene $S = [0]$. Los hijos del nodo 6 son 4 , 1 y 5 , todos ellos ocupados. Sus valores son 20 ,10 y 20 , respectivamente. Al ordenarlos en orden no decreciente se obtiene $c[6] = [1, 5, 4]$ (en en cuenta que $c[6] = [1, 4, 5]$ también sería válido, ya que los nodos 4 y 5 tienen el mismo valor). A continuación, el procedimiento llama a **traverse** en cada nodo de $c[6]$ en ese orden.

- **traverse(1)** : el nodo 1 tiene el valor 10 ; al añadirlo a S se obtiene $S = [0, 10]$. El nodo 1 no tiene hijos ocupados, por lo que esta llamada finaliza.
- **traverse(5)** : el nodo 5 tiene el valor 20 ; al añadirlo a S se obtiene $S = [0, 10, 20]$. El nodo 5 tiene un hijo ocupado, el nodo 2 , por lo que $c[5] = [2]$ y el procedimiento llama a **traverse(2)** .
 - **traverse(2)** : el nodo 2 tiene el valor 30 ; al añadirlo a S se obtiene $S = [0, 10, 20, 30]$. El nodo 2 no tiene hijos ocupados, por lo que esta llamada finaliza.
 - La ejecución vuelve a **traverse(5)** , que ya ha procesado todos los hijos de $c[5]$, por lo que su llamada finaliza.
- **traverse(4)** : el nodo 4 tiene el valor 20 ; al añadirlo a S se obtiene $S = [0, 10, 20, 30, 20]$. El nodo 4 no tiene hijos ocupados, por lo que esta llamada finaliza.

Todas las llamadas han finalizado y no quedan más nodos por procesar. Por último, se retiran todas las bolas de la máquina y **collect** devuelve el arreglo $S = [0, 10, 20, 30, 20]$.

Tu tarea consiste en determinar el número de nodos N y devolver un arreglo de padres $R = [R[0], R[1], \dots, R[N - 2]]$ que describa la estructura de la máquina, pero con una etiquetación potencialmente diferente de los nodos que no sean hojas ni la raíz. Formalmente, se considera correcto un arreglo R si es posible asignar una etiqueta única $L[u]$ con $0 \leq L[u] < N$ a cada nodo u de la máquina, de tal manera que:

- $L[u] = u$ para todos los $0 \leq u < M$ y $u = N - 1$, y
- $L[P[u]] = R[L[u]]$ para todos los $0 \leq u < N - 1$.

No es necesario que $R[u] > u$. La máquina **debe estar vacía** cuando entregues tu solución.

Sea K el número de operaciones **collect** realizadas, y B el valor máximo escrito en cualquier bola en todas las llamadas **insert** . Sea $C = K + B$. Entonces, C **no debe superar** 1000 . Tu puntuación en algunas subtareas depende del valor de C .

Detalles de la implementación

Debes implementar el siguiente procedimiento.

```
std::vector<int> find_structure(int M)
```

- M : el número de hojas de la máquina.
- El procedimiento se llama exactamente una vez por cada caso de prueba.
- El procedimiento debe devolver una matriz $R = [R[0], R[1], \dots, R[N - 2]]$ de longitud $N - 1$, una matriz correcta de padres.

Para interactuar con la máquina, tu procedimiento puede llamar a los dos procedimientos siguientes.

El primer procedimiento es:

```
bool insert(int U, int X)
```

- U : el índice de la hoja en la que debe insertarse la bola. Debe cumplirse la condición $0 \leq U < M$.
- X : el valor entero de la bola. Debe cumplirse la condición $0 \leq X \leq 1000$.
- Este procedimiento devuelve **true** si la bola se ha insertado correctamente, o **false** si la hoja U ya estaba ocupada.
- Este procedimiento puede invocarse como máximo 500 000 veces.

El segundo procedimiento es:

```
std::vector<int> collect()
```

- Recoge los valores de todas las bolas insertadas, vacía la máquina y devuelve el arreglo recopilado S .

Si en cualquier momento durante la ejecución de tu programa el valor de C supera 1000 , tu solución recibirá el veredicto `Output isn't correct: Too many resources used.`

La estructura de la máquina se fija antes de que se llame a **find_structure**. El grader es **determinístico** en el sentido que si tu lo ejecutas dos veces y ambas ejecuciones corren la misma secuencia de operaciones, las llamadas a **collect** retornarán los mismos arreglos.

Restricciones

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Subtareas

Subtarea	Puntuación	Restricciones adicionales
1	5	La raíz tiene exactamente M hijos.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Sin restricciones adicionales.

En las subtareas 3 y 4, tu puntuación depende del valor de C de la siguiente manera.

Subtarea 3 (25 puntos)

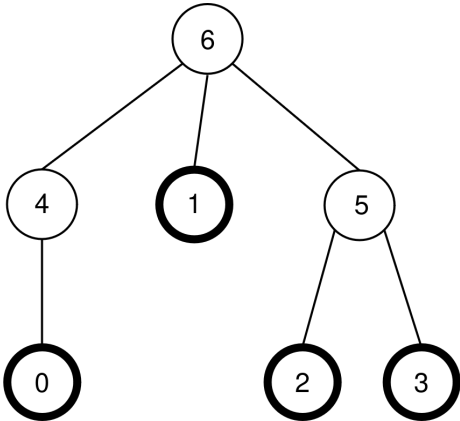
Límites	Puntuación
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Subtarea 4 (60 puntos)

Límites	Puntuación
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Ejemplo

Consideremos la misma máquina que en la descripción de la tarea, es decir, $N = 7$, $M = 4$ y $P = [4, 6, 5, 5, 6, 6]$. La máquina se muestra de nuevo en la siguiente figura:



El grader realiza la siguiente llamada:

```
find_structure(4)
```

El procedimiento puede realizar la siguiente secuencia de llamadas:

1. `insert(0, 0)` : la hoja 0 está libre, por lo que se coloca una bola con el valor 0 en la hoja 0 . El nodo $P[0] = 4$ está libre, por lo que la bola se desplaza al nodo 4 . El nodo $P[4] = 6$ está libre, por lo que la bola se desplaza al nodo 6 . Dado que el nodo 6 es la raíz, el movimiento se detiene. La llamada devuelve `true` .
2. `insert(3, 20)` : la hoja 3 está libre, por lo que se coloca una bola con el valor 20 en la hoja 3 . El nodo $P[3] = 5$ está libre, por lo que la bola se desplaza al nodo 5 . Dado que $P[5] = 6$ está ocupado, el movimiento se detiene. La llamada devuelve `true` .
3. `insert(1, 10)` : la hoja 1 está libre, por lo que se coloca una bola con el valor 10 en la hoja 1 . Dado que $P[1] = 6$ está ocupado, la bola permanece en el nodo 1 . La llamada devuelve `true` .
4. `insert(2, 30)` : la hoja 2 está libre, por lo que se coloca una bola con el valor 30 en la hoja 2 . Dado que $P[2] = 5$ está ocupado, la bola permanece en el nodo 2 . La llamada devuelve `true` .
5. `insert(1, 25)` : la hoja 1 está ocupada, por lo que la bola no se coloca en la hoja 1 . La llamada devuelve `false` .
6. `insert(0, 20)` : la hoja 0 está libre, por lo que se coloca una bola con el valor 20 en la hoja 0 . El nodo $P[0] = 4$ está libre, por lo que la bola se desplaza al nodo 4 . Dado que $P[4] = 6$ está ocupado, el movimiento se detiene. La llamada devuelve `true` .

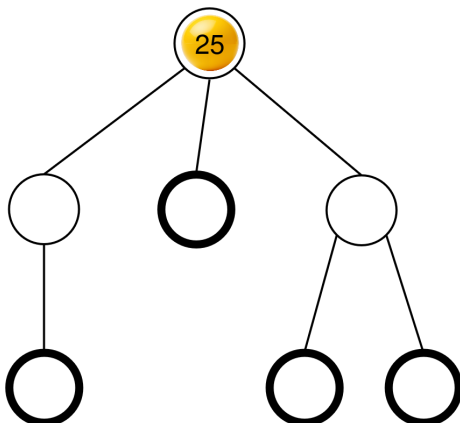
La configuración resultante de las bolas ya se mostró en la descripción de la tarea.

A continuación, el procedimiento puede llamar a:

```
collect()
```

La ejecución de esta llamada ya se ha explicado en la descripción de la tarea, por lo que esta llamada devuelve $S = [0, 10, 20, 30, 20]$. A continuación, la máquina vuelve a estar vacía.

A continuación, el procedimiento puede llamar a `insert(2, 25)` . La hoja 2 está libre, por lo que se coloca una bola con el valor 25 en la hoja 0 . A continuación, esta bola se desplaza al nodo 5 y, después, al nodo 6 , donde se detiene. La llamada devuelve `true` . La configuración resultante de las bolas se muestra en la siguiente figura.



Por último, el procedimiento puede llamar a `collect()` , que devuelve $S = [25]$ y vacía la máquina.

Hay dos matrices principales correctas que `find_structure` puede devolver: $R = P = [4, 6, 5, 5, 6, 6]$ (que se corresponde con el etiquetado $L = [0, 1, 2, 3, 4, 5, 6]$) y $R = [5, 6, 4, 4, 6, 6]$ (que se corresponde con el etiquetado $L = [0, 1, 2, 3, 5, 4, 6]$). En este ejemplo, $K = 2$ y $B = 30$, por lo que $C = 32$.

Grader de ejemplo

Formato de entrada:

N M

P[0] P[1] P[2] ... P[N-2]

Formato de salida:

```
K B C
```

```
Q
```

```
R[0] R[1] R[2] ... R[Q-1]
```

Aquí, Q es la longitud del arreglo R devuelta por `find_structure`.